python

A practical, hands-on introduction to Python 3 that gets students coding quickly. The course covers installation and editor setup, using the REPL and running scripts, virtual environments, and core language fundamentals: syntax, variables and built-in types, with short exercises and small projects to practice each concept.

Audience: student Template: Technical Chapters: 2

WHAT YOU WILL BE ABLE TO DO

- Install and configure Python 3 and a suitable editor on Windows, macOS, or Linux
- Create and activate a project virtual environment (venv) and run Python scripts from the REPL and command line
- Read and write Python code demonstrating correct syntax, indentation, variable assignment and naming conventions
- Use core built-in types (int, float, bool, str) and format strings using f-strings in small programs
- Debug common setup and syntax errors and apply basic best practices for beginner Python projects

python

What's in this course

1

1. Course overview & setup

This chapter helps you install Python 3, pick an editor, and run code both in the REPL and from a script. You learn to…

2

2. Python basics: syntax, variables & types

You learned how Python uses indentation to define blocks and why consistent 4-space indents matter. Yo…

1. Course overview & setup

A smooth setup lets you focus on learning Python, not fighting tools. In this chapter you'll install Python 3, pick a beginner-friendly editor, and learn two ways to run code: interactively in the REPL and from a .py script. You'll also create a per-project virtual environment (venv) so experiments don't affect other projects or your system Python.

We'll use concrete commands that work on Windows, macOS, and Linux. You'll verify your version, understand the Python interpreter vs. scripts, and see how your editor calls the interpreter behind the scenes. By the end, you'll have a reproducible workflow you can use for the rest of the course and beyond.

If you already have Python, this chapter still matters: it helps you confirm which interpreter your editor and terminal use, avoid PATH mix-ups, and isolate dependencies with venv. Five quick steps and you'll be writing and running real code with confidence.

LEARNING OBJECTIVES

- Verify a working Python 3 installation and identify the active interpreter
- Install Python 3 on Windows/macOS/Linux when missing
- Choose and configure an editor (VS Code or PyCharm) to use the right interpreter
- Create, activate, and use a per-project virtual environment (venv)
- Run code in the REPL and from a .py script using terminal commands
- Diagnose common setup issues (PATH, multiple versions, venv not activated)

Core concepts: interpreter, scripts, REPL, venv, and PATH

Python interpreter: the program that executes your code (e.g., python, python3, or py on Windows). A script is a .py file that the interpreter runs from top to bottom, like python hello.py. Editors don’t replace the interpreter—they invoke it. Keeping these roles straight avoids confusion: you edit in the editor; you run with the interpreter.

REPL (Read–Eval–Print Loop): an interactive session you start by running python or python3 (or py on Windows) without a filename. You type an expression or statement; Python evaluates it and prints the result immediately. The REPL is perfect for trying ideas quickly, exploring library behavior, or debugging small snippets without creating a file.

Virtual environment (venv): a folder that contains its own interpreter references and installed packages for a single project. Activating it ensures python and pip point to that project's environment. PATH is the shell's search list for executables; if Python isn't on PATH (or a different Python appears earlier), commands may run the wrong interpreter. On Windows, the py launcher can select a specific Python 3 runtime (e.g., py -3).

You'll use these terms constantly: - Interpreter: runs code - Script (.py): code in a file - REPL: interactive Python prompt - venv: per-project environment - PATH: where the shell looks for programs - py (Windows): chooses among installed Pythons

How it works: from editor to interpreter to output

Your editor saves code to a .py file or sends it directly to the interpreter. The shell resolves which python to run using PATH (or the py launcher on Windows). If a venv is activated, python and pip come from that environment, so imports and installs stay isolated. The interpreter executes your code and writes results to standard output (your terminal's console). Failures usually trace back to: the wrong interpreter selected, venv not activated, or Python not on PATH.

OS	Check version	Create venv	Activate venv
Windows	py --version or py -3 --version	py -3 -m venv .venv	.venv\Scripts\activate
macOS	python3 --version	python3 -m venv .venv	source .venv/bin/activate
Linux	python3 --version	python3 -m venv .venv	source .venv/bin/activate

Windows tip: py -0 lists installed Python versions. Use py -3 to target your default Python 3. This avoids PATH pitfalls when multiple versions are present.

Visual unavailable
Show the execution pipeline and where venv and PATH influence which interpreter runs.

Execution pipeline: the interpreter runs REPL or scripts; PATH/py choose the interpreter; venv isolates packages.

Failure modes: If python opens the wrong version, verify PATH or use py on Windows. If imports fail after installs, ensure the venv is active and that you installed with python -m pip (so pip matches the interpreter). If the editor runs a different interpreter than your terminal, use the editor’s “Select Interpreter” to point it at your venv.

Visual unavailable
Prime the comparison that follows: two ways to run Python, side-by-side.

Two paths: interactive REPL vs. running a script file.

Visual unavailable

Compare REPL and script runs at a glance: when to use each and typical commands.

REPL vs. script: fast experiments vs. repeatable programs.

Let’s create a tiny program you can run anywhere. You’ll see your Python version and whether a venv is active. Save this as hello.py inside a new project folder (create and activate a venv first if you want). Then run it from the terminal using python hello.py (or py -3 hello.py on Windows).

PYTHON CODE

```
import sys

print("Hello, Python!")
print("Python version:", sys.version.split()[0])
print("venv active:", sys.prefix != sys.base_prefix)
```

OUTPUT

```
Hello, Python!
Python version: 3.12.15
venv active: False
```

This program prints a greeting, your interpreter’s version, and whether you’re inside a virtual environment. If you activate a venv and run again, the venv line should switch accordingly. This is a quick smoke test to confirm that your terminal and editor are using the interpreter you expect.

New learners often hit the same setup snags. Use the symptoms below to quickly pinpoint the cause, then apply the fix. In most cases, selecting the correct interpreter or activating the venv resolves the issue immediately.

COMMON MISTAKE

Symptom: python runs but imports fail after installing packages. Cause: pip installed to a different interpreter than the one you're running. Fix: always install with python -m pip install <pkg> using the same python you'll run, and ensure your venv is activated before installing.

COMMON MISTAKE

Symptom: On Windows, python opens the Microsoft Store or the wrong version runs. Cause: PATH/app alias confusion or multiple Pythons installed. Fix: prefer py -3 to select Python 3; verify py -0; ensure your editor's selected interpreter points to your project venv.

Best practices for a reliable Python setup

Keep one venv per project, and activate it before running or installing anything. Use python -m pip rather than pip so the installer matches your interpreter. On Windows, use py -3 to avoid PATH surprises. In editors, explicitly Select Interpreter to bind the workspace to your venv. Verify with python --version (or py --version) and a quick REPL test.

PRO TIP

Quick checklist you can reuse: - Create project folder; python -m venv .venv (or py -3 -m venv .venv on Windows) - Activate venv; confirm python --version - Install with python -m pip install <pkg> - In your editor, Select Interpreter -> .venv - Run hello.py; confirm expected version and venv status

LEARNING OBJECTIVES

- Create one venv per project and add the venv directory to .gitignore (do not check the venv into version control)
- Use python -m pip to ensure pip matches the active interpreter
- Verify interpreter selection in both terminal and editor

- Prefer py -3 on Windows to target Python 3 reliably
- Smoke-test with a tiny script to confirm venv and version

EXERCISE · EASY

Set up your first course project and prove it works end-to-end.

1. Create a folder named quickstart and open a terminal in it.
2. Create a venv: Windows: py -3 -m venv .venv; macOS/Linux: python3 -m venv .venv.
3. Activate it: Windows: .venv\Scripts\activate; macOS/Linux: source .venv/bin/activate.
4. Run python --version (or py --version) and python -m pip --version.
5. Create hello.py with the Technical Example code and run it.
6. Open the folder in VS Code or PyCharm and select the .venv interpreter, then run hello.py from the editor.

Hints:

- If command not found, open a new terminal after installation so PATH refreshes.
- In VS Code, use the Command Palette: Python: Select Interpreter.
- If imports fail, confirm venv activation and use python -m pip install <pkg>.

Expected outcome: hello.py runs from terminal and editor, prints the expected Python version, and reports venv active: True when the environment is activated.

KNOWLEDGE CHECK

1. What does the Python interpreter do in your workflow?

A. It edits your code automatically

B. It executes your code (from REPL or scripts)

C. It installs packages globally only

D. It configures your editor settings

Answer: It executes your code (from REPL or scripts) — The interpreter is the runtime that executes code you send via REPL or .py scripts.

2. Why use a virtual environment (venv) per project?

A. To speed up your CPU

B. To isolate packages and interpreter references per project

C. To replace the editor

D. To avoid writing import statements

Answer: To isolate packages and interpreter references per project — A venv keeps dependencies and interpreter references local to the project, preventing conflicts.

3. On Windows, which command reliably selects a Python 3 interpreter?

A. python3

B. py -3

C. pip3

D. pythonx

Answer: py -3 — The py launcher can select among installed versions; py -3 targets a Python 3 interpreter.

4. After installing a package, your script still says ModuleNotFoundError. What should you check first?

A. Whether your editor theme is dark mode

B. That you installed with python -m pip using the same interpreter you run

C. That you saved the file as .txt

D. That you disabled your firewall

Answer: That you installed with python -m pip using the same interpreter you run — If pip and python point to different interpreters, installs won't be visible to the one running your code.

SUMMARY

- You installed or verified Python 3 and learned how to check the active interpreter.
- You can run code in the REPL and from a script, and you know when to use each.
- You created and activated a venv so packages and interpreter references stay per-project.
- You configured your editor to use the project’s interpreter.
- You can diagnose common issues like PATH mix-ups and mismatched interpreters.

Next

- Move on to Python basics: syntax, indentation, variables, and built-in types.
- Keep your quickstart project open; we’ll add small programs to practice fundamentals.

2. Python basics: syntax, variables & types

Introduction

This chapter gets you fluent with Python’s building blocks: correct indentation, clean variable names, and the essential data types you’ll use everywhere. Mastering these early prevents hard-to-read code and frustrating errors, so you can focus on solving problems rather than fighting the interpreter.

You’ll see how names bind to objects, not to fixed types, and how to format readable output with f-strings. We’ll keep examples short and runnable. By the end, you’ll be able to write small, tidy scripts that do real work and set you up for control flow and functions later.

Visual unavailable
Anchor the goals on variables, indentation, and basic types without using text.

We’ll work with ints, floats, bools, and strings while keeping indentation and naming clean.

LEARNING OBJECTIVES

- Apply Python indentation rules to create correct code blocks
- Bind and rebind variables to values and explain that names do not have fixed types
- Use int, float, bool, and str in small calculations and decisions
- Format readable output with f-strings, including basic format specifiers
- Avoid common pitfalls like mixing tabs and spaces or trying to mutate strings

Core concepts: indentation, names, and types

Indentation is the whitespace at a line’s start that groups statements into blocks. Python uses indentation instead of braces to mark scope. The PEP 8 style guide recommends four spaces per indent level and warns against mixing tabs and spaces. Consistent indentation makes structure obvious: lines with the same indent belong together; deeper indent means a nested block.

A variable is just a name bound to an object. Assignment picks a value and binds the name to it; it does not declare a type. You can rebind a name to a different kind of object later. Objects carry types like int, float, bool, and str. Some types are immutable (strings), meaning you create new values instead of changing them in place.

KEY DEFINITIONS

Indentation: whitespace that defines blocks (PEP 8: 4 spaces). Variable (name): a name bound to an object. Mutable vs Immutable: lists/dicts change in place; strings/tuples don’t. f-string: a string prefixed with f that evaluates expressions inside { } (since Python 3.6).

How it works: blocks, statements, and line continuation

Python reads your file line by line. A line ending with a colon (such as after if, for, while, def, class) starts a new block, and the next line must be indented deeper. Lines at the same indentation level belong to the same block; when indentation decreases, the previous block ends. You can continue a long expression across lines either with a trailing backslash or implicitly inside (), [], or {}.

Visual unavailable
Show the ordered steps Python follows to group lines into blocks using indentation.

Indentation flow: colons begin suites; increased indent enters a block; dedent closes one or more blocks.

Concept	Example	Notes
Explicit continuation	total = a + \ b + c	Backslash continues a statement to the next line
Implicit continuation	nums = [1, 2, 3]	Inside (), [], {} you can break lines freely
Block start	if x > 0: print(x)	Colon starts a suite; next line must indent
Dedent	print("done")	Outdenting closes the previous block(s)

Visual unavailable
Introduce the idea of two categories before the labeled comparison below.

Some types are immutable, others are mutable. Know which you have before modifying.

Visual unavailable
Compare immutable and mutable core types with concrete labels.

Immutable values are replaced by new ones; mutable values can be changed in place.

Technical example: small script using variables and f-strings

This example shows variable binding, arithmetic, a boolean expression, and f-strings. It keeps the code compact but expressive, and it demonstrates that names can be rebound and types are attached to values. Run it as a single script to see formatted, readable output without needing any external input or packages.

```python
name = "Avery"
age = 19
next_age = age + 1
pi = 3.14159
is_adult = age >= 18

# f-strings for readable output
print(f"Hello, {name}! Next year you'll be {next_age}.")

radius = 2
area = pi * (radius ** 2)
print(f"Circle area (r={radius}): {area:.2f}")

# Strings are immutable; this creates a new string
banner = "python"
shouted = banner.upper()
print(f"Original: {banner} | Uppercase: {shouted}")
print(f"Adult? {is_adult}")
```

Notice how each print uses an f-string for clarity. The radius and area are shown with formatting (.2f rounds to two decimals). The .upper() call returns a new string, leaving the original unchanged—evidence of string immutability. Also observe that booleans come from expressions like age >= 18 and print as True or False.

Visual unavailable

Prime the learner to watch out for tab/space issues and string mutation attempts.

Tabs vs spaces and trying to mutate strings are frequent beginner pitfalls.

FREQUENT MISTAKES AND QUICK FIXES

- Mixing tabs and spaces: causes IndentationError or odd alignment. Fix by converting all indentation to 4 spaces. - Assuming assignment declares a type: names have no fixed type; the object does. - Trying to mutate strings: s[0] = 'x' raises TypeError; build a new string instead. - Using 'python' on systems where it is absent or not Python 3: invoke 'python3' (or the Windows 'py' launcher) to run Python 3 code.

If your editor shows mixed indentation, use its “convert indentation” feature and enable visible indent guides. When you need to change text, prefer methods that return new strings (replace, upper, slice and concatenate). For command-line execution, many Unix-like systems expect python3 explicitly; Windows commonly provides the py launcher (e.g., py -3).

Best practices for clean, beginner-friendly code

Visual unavailable

Convey the idea of a practical checklist for indentation, naming, and formatting.

Keep a short checklist: spaces for indents, good names, readable lines, and f-strings for output.

ACTIONABLE HABITS

- Use 4 spaces per indent; never mix tabs and spaces. - Prefer snake_case for variable names; make them descriptive at non-trivial scope. - Keep lines reasonably short; break expressions inside (), [], {}. - Use f-strings for output; apply format specifiers like :.2f when needed. - Treat strings as immutable; construct new values rather than in-place edits.

Name things for what they represent (total_price, max_speed). Break up long expressions rather than letting them run off the screen. When formatting, put the intent in the string: f"{value:.1f} kg" is self-documenting. Configure your editor to insert spaces on Tab and to display invisible characters so indentation mistakes are obvious.

Visual unavailable

Signal hands-on practice with multiple small tasks to complete.

Three quick exercises: predict, fix, and write.

EXERCISE · EASY

Complete three mini tasks in a single file or separate cells.

1. Predict the outputs, then run and check: a = 3; b = 2.5; print(a + b); print(f"{a}{b}")
2. Fix this broken script: name = "Kai"; print(f"Hi, {name}"); print(name[0] = 'C') # Make it print 'Cai' without exceptions
3. Write a tiny script: ask for a person and a value (hardcode variables), compute value * 1.2, and print with an f-string rounded to 1 decimal.

Hints:

- Numbers add; strings placed next to each other inside an f-string concatenate as text.
- Strings are immutable; rebuild with slicing: 'C' + name[1:]
- Round with format specifier: f"{amount*1.2:.1f}"

Expected outcome: You can explain each result, repair code that mutates a string by creating a new one, and produce a clear, formatted printout using f-strings.

Visual unavailable

Introduce a slightly more open-ended task involving numeric formatting.

A small calculator challenge: combine variables, booleans, and formatting.

Visual unavailable

Cue a short knowledge check about indentation, names, and f-strings.

Quick check: apply what you learned.

KNOWLEDGE CHECK

1. In Python, what determines where a code block starts and ends?

A. Curly braces { }

B. Keywords begin/end

C. Indentation level changes

D. Semicolons ;

Answer: Indentation level changes — A colon starts a suite and deeper indentation begins the block; dedenting closes it. Python does not use braces or begin/end.

2. What does assignment do in Python?

A. Declares a variable’s static type

B. Binds a name to an object

C. Creates a constant

D. Reserves memory at a fixed address

Answer: Binds a name to an object — Names have no fixed type. You can rebind a name to different objects; the object itself carries the type.

3. Which f-string prints pi to three decimals when pi = 3.14159?

A. f"{pi:3}"

B. f"{pi:.3f}"

C. f"{pi:3.}"

D. f"{pi:3d}"

Answer: f"{pi:.3f}" — .3f formats a floating-point value with three digits after the decimal point.

4. Why does s[0] = 'X' fail when s is a str?

A. Strings are 1-indexed

B. Only uppercase letters are allowed

C. Strings are immutable

D. Indexing is not supported on strings

Answer: Strings are immutable — str objects cannot be changed in place. Build a new string, e.g., 'X' + s[1:].

Visual unavailable

Convey wrapping up the essentials: indentation, names, and core types.

You now have the essentials to write tidy, readable Python.

SUMMARY

- Indentation defines blocks; use 4 spaces and avoid tabs.
- Assignment binds names to objects; names have no fixed types.
- Work fluently with int, float, bool, and str; remember strings are immutable.
- Use f-strings for clear, concise output, with format specifiers like :.2f.
- Fix common issues quickly: convert tabs to spaces, rebuild strings instead of mutating.

Next

- Apply these basics to branching and loops to control program flow.
- Practice by writing tiny scripts that compute, compare, and format results.